

Understanding FAT 12

You need to address many details to solve this problem. The exercise is broken down into parts to reduce the overall complexity of the problem:

- Part A: Construct the command to list directory contents
- Part B: Change the current directory definition
- Part C: Remove a file

This section provides you with a good start on considering all of them, but you will have to discover and resolve many on your own.

The MS-DOS Disk Format

MS-DOS defines a particular format for floppy disks. The MS-DOS BIOS (Basic Input/Output System) provides a set of programs that can read and write disk sectors. It also provides an additional abstraction on the disk address called the logical sector address. Logical sector 0 corresponds to the sector at surface 0 (on a hard disk), track 0, sector 1 of the floppy disk. (Surfaces and tracks are numbered from zero, but sectors are numbered from 1; the reasons for this have long been forgotten.) Logical sector 1 is at surface 0, track 0, sector 2, and so on.

Before a floppy disk can be used for normal I/O operations, it must be formatted to contain essential information in pre-specified locations. Specifically, logical sector 0 contains a reserved area (also called the boot sector or boot record). The MS-DOS boot sequence relies on the boot sector's being located at logical sector 0 and organized as shown in Figure 2.14. Several of the fields will not make sense until you have had a chance to read the details about the FAT and how it uses the boot record information. Hard disks also may be partitioned (see Part 1, Section 3.5), with each partition defining a file system.

Figure 2.15 shows the organization of an MS-DOS-formatted floppy disk. The FAT generally is replicated on the disk, so location 0x10 in the boot sector tells the system how many copies of the FAT are formatted onto this disk (usually two). The FAT is replicated so that if the first copy is accidentally destroyed, then the disk can be recovered by copying the second copy over the first. Each root directory entry uses 32 bytes. Location 0x11 in the boot sector indicates how many entries are in the disk (1.44MB floppy disks have space for 224 directory entries, using 14 sectors). Each floppy disk drive has a fixed geometry that specifies the fundamental parameters for that drive. These include the number of

- bytes per disk block (a block is usually called sector in the disk drive literature),
- sectors in each track,
- read/write heads, and
- cylinders (tracks).

A floppy disk is formatted with the geometry in its boot sector. Logical sectors are numbered from zero, even though individual sectors on a track are typically numbered from 1. The logical sector 1 on track T, head H, sector S is located at

0x00	0x02	<a jump instruction to 0x1e>
0x03	0x0a	Computer manufacturer's name
0x0b	0x0c	Bytes per sector
0x0d	0x0d	Sectors per cluster
0x0e	0x0f	Reserved sectors for the boot record
0x10	0x10	Number of FATs
0x11	0x12	Number of root directory entries
0x13	0x14	Number of logical sectors
0x15	0x15	Medium descriptor byte (used only on old versions of MS-DOS)
0x16	0x17	Sectors per FAT
0x18	0x19	Sectors per track
0x1a	0x1b	Number of surfaces (heads)
0x1c	0x1d	Number of hidden sectors
0x1e	..	Bootstrap program

Figure 2.14
Boot Sector

Logical Sector	Content
0	Boot Sector
1	First Sector in the (first) FAT
...	
10	First sector in the second FAT (if there is a second one; see 0x10 in the boot sector)
19	First sector in the floppy disk's root directory
xx	Last sector in the root directory (see 0x11 in the boot sector)
xx+1	Beginning of data area for the floppy disk

Figure 2.15
Floppy Disk Organization

$$l = S - 1 + T * (\text{heads} * \text{sectorsPerTrack}) + H * \text{sectorsPerTrack}.$$

The MS-DOS FAT

The floppy disk format is based on a FAT file system in which the media (floppy disk) is divided into a reserved area (containing the boot program), the actual allocation tables, a root directory, and file space; see Figure 2.15. The second copy of the FAT is the same as the first copy; it is used to recover the disk if the first copy of the FAT is accidentally destroyed. Space allocated for files is represented by values in the allocation table, thereby effectively providing a linked list of all of the blocks in the file. Special values designate EOF, unallocated blocks, and bad blocks. The original FAT had many limitations. It had no subdirectories and was limited to very small disks. In addition, recovering the disk if the allocation tables were damaged was very difficult.

During the time of the explosive growth of the personal computer, the size of disk drives also increased dramatically. At the same time, different variants of FAT were derived to accommodate larger disks. The basic FAT organization (see Figure 2.16) differs among disk types as follows:

- By the size of the entries (M in the figure can be 12 or 16 but can be 12, 16, or 32 in Windows 98 and Windows 2000)
- By the number of actual tables
- By the size of the logical sector addressed by a FAT entry

In the simplest variant, one FAT entry corresponds to each sector on the disk. A file is a set of disk sectors, with the FAT entry corresponding to the first entry designating the logical sector number of the second block. Similarly, the FAT entry of the second block specifies the logical sector number of the third block, and so on. The FAT entry for the last block contains an EOF designator. Thus the FAT is a linked list of disk sectors. If you know the address of the first sector, i —and therefore the index into the FAT—you can use the FAT to reference the next logical sector in the file (see the figure). The content, j , of FAT index i is a logical sector number, as well as an index to the second FAT entry for the file.

As disk sizes grew larger than 32MB, the FAT organization began to use the notion of a cluster of sectors. A cluster is a group of contiguous sectors that are treated as a virtual sector within the FAT. In contemporary implementations of the FAT file system, the FAT entry addresses a cluster rather than an individual disk sector. With the use of clusters, the FAT organization can address groups of, say, four sectors as if they were a single sector that contained four times as many bytes as a single sector. This means that disk space is allocated to a file on a cluster-by-cluster basis. However, a 12-bit FAT is still limited to 128MB.

Today, floppy disks use 12-bit FATs and hard disks use 16-bit or 32-bit FATs.

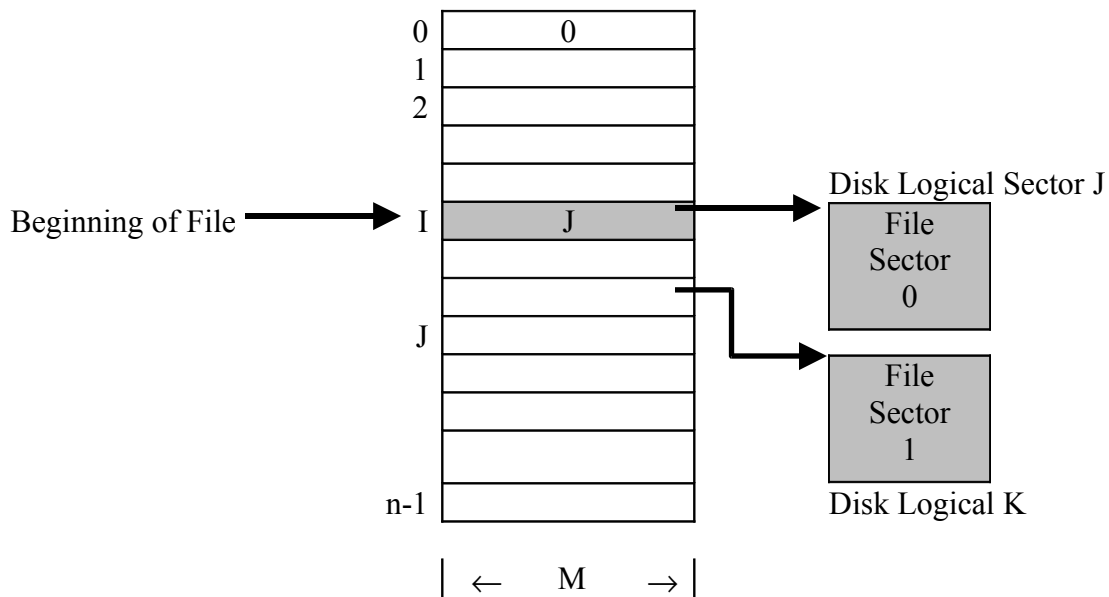


Figure 2.16
FAT Organization

Directories and File Storage

Information about files is stored in a directory. You can look up information about a file (its size, its location on the floppy disk, the time it was created, and so on) by using the file's name. FAT-12 (the version of FAT with 12-bit FAT entries) uses a simple list of fixed-size directory entries. A directory entry can be found in a directory by searching linearly through the disk sectors that contain the directory list. That is, traversing the list of directory entries in a directory involves looking at multiple disk sectors and then at multiple entries within each sector.

A directory entry contains the filename and a description of the location of the file's data. The entry also contains the file's size in bytes (in case the file size is not an exact multiple of the sector size). Directory entries use the same format whether they are in the root directory or in a subdirectory. Each entry is 32 bytes long, thereby allowing 16 entries per standard 512-byte sector. The root directory has a fixed maximum number of entries (the number is stored in the boot sector) and occupies a contiguous group of logical sectors, as determined by the format. By contrast, subdirectories are stored in a set of sectors that are managed in the same manner as files. That is, logically contiguous sectors are not necessarily physically contiguous on the disk, and therefore they must be accessed by using the FAT.

Figure 2.17 shows the layout for a directory entry. All multi-byte integers are in little-endian order, meaning that the least significant byte is stored first.

The filename and extension are stored as uppercase ASCII characters. Invalid entries have names beginning with 0x00 (the entry has not been used before) or 0xe5 (the entry was used before but has been released). The starting cluster number is slightly deceiving. Although it references the starting cluster (sector) number, it cannot reference sectors used for the boot record, FAT copies, or root directory. A starting cluster number k actually refers to logical sector number $31 + k$.

Offset	Length	Description
0x00	8	Filename
0x08	3	Extension
0x0B	1	Bit field for attributes
0x0C	10	Reserved
0x16	2	Time (coded as $\text{Hour} * 2048 + \text{Min} * 32 + \text{Sec} / 2$)
0x18	2	Date (coded as $\text{Year} - 1980 * 512 + \text{Month} * 32 + \text{Day}$)
0x1A	2	Starting Cluster Number
0x1C	4	File size (in bytes)

Figure 2.17
Directory Entry

Bit	Mask	Attribute
0	0x01	Read-only
1	0x02	Hidden
2	0x04	System
3	0x08	Volume Label
4	0x10	Subdirectory
5	0x20	Archive
6	0x40	Unused
7	0x80	Unused

Figure 2.18
Directory Entry Attributes

The attribute byte stores bits for attributes, similar to UNIX attributes. The bit fields are shown in Figure 2.18. Note that bit 0 is the least significant bit. A bit set to 1 means that the file has that attribute and 0 means that it does not. So, for example, a file with attributes `0x20 == 00100000b` has the archive bit set and all others cleared. A hidden, read-only subdirectory would be `00010011b == 0x13`.

Clusters and File Storage

Files on FAT disks are divided into clusters of N sectors. N ranges from 1 to 32, depending on the format (hard disks use large clusters, so their allocation tables do not get too large). To access a file, you need to determine the list of clusters that it occupies. For example, a 2K file FOO.TXT might occupy clusters 34, 19, 81, and 47. Because the directory has a fixed size, there is no room to store an arbitrary list of clusters in the directory entry. Instead, the system stores only the first cluster; the FAT is used to link the rest. For example, to find out what cluster follows cluster C , use the C -th entry in the FAT. So for FOO.TXT, the system would store 34 in the directory entry for the first cluster.

```
FAT[34] == 19
FAT[19] == 81
FAT[81] == 47
FAT[47] == 0xff8
```

The special value `0xff8` marks the last cluster in the file. Any value from `0xff8` through `0xffff` may be used for this purpose. Use of this value is redundant because the file length indicates how many clusters there are, but the redundant information helps in recovering damaged disks.

More on the FAT

The first two entries of the FAT (for clusters 0 and 1) are reserved. The first one holds the media descriptor, which, for floppy disks, takes on the values shown in Figure 2.19. The FAT space, for clusters with two or more sectors, can contain the values for an entry, as shown in Figure 2.20.

FAT Packing

The data structure would be easier to manipulate if the cluster numbers in the FAT were stored in two-byte groups in the FAT sectors. But early MS-DOS machines were limited in space, so designers

decided to pack two 12-bit FAT entries into 3 bytes. Thus bytes $3k$ through $3k + 2$ contain elements $2k$ and $2k + 1$ of the FAT array. In particular:

- Byte $3k$ contains the least significant 8 bits of $\text{FAT}[2k]$.
- The least significant 4 bits of byte $3k + 1$ contain the most significant 4 bits of $\text{FAT}[2k]$.
- The most significant 4 bits of byte $3k + 1$ contain the least significant 4 bits of $\text{FAT}[2k + 1]$.
- Byte $3k + 2$ contains the most significant 8 bits of $\text{FAT}[2k + 1]$.

As an example of how entries are packed into the FAT, suppose that a short file has been written to a new 1.44MB floppy disk. This new file begins at the first part of the file space, in cluster 2. The rest of its sectors follow contiguously (because the disk was not already fragmented when it was written). The beginning of the FAT now appears as shown in the hexadecimal dump in Figure 2.21. The first 3 bytes (F0 FF FF) are the two reserved entries and can be ignored. The first entry corresponding to the example file is that for cluster 2, its beginning block. From the algorithm, the entry for cluster 2 is obtained by combining parts of bytes at offsets 3 and 4. This gives 0x003, since the first byte is less significant. This means that the FAT entry for cluster 2 is a pointer to its next cluster, number 3. To fetch number 3 then, you use the bytes at offsets 4 and 5, thereby resulting in 0x004. So the FAT entry for cluster 3 points to the next cluster in the file, number 4. Similarly, cluster 4 has 0x005 for the next cluster, and so on. Finally, at cluster 9 ($2 * 4 + 1$), reading the entries at offsets 13 and 14 ($3 * 4 + 1$ and $3 * 4 + 2$) gives 0xffff, a reserved value that indicates the last cluster in a file. So cluster 9 marks the end of the file that began at cluster 2. The remaining clusters, from 10 up, are still unallocated and so have the reserved value 0.

Description	Heads	Sectors/ Track	Cylinders	Capacity
0x0fe	1	8	40	160K
0x0ff	2	8	40	320K
0x0fc	1	9	40	180K
0x0fd	2	9	40	360K
0x0f9	2	9	80	720K
0x0f0	2	18	80	1.44M

Figure 2.19
FAT Entry Values

Value	Meaning
0x000	Unused
0xFF0-0xFF6	Reserved cluster
0xFF7	Bad Cluster
0xFF8-0xFF	Last cluster in a file
(anything else)	Number of the next cluster in the file

Figure 2.20
FAT Space Values

Figure 2.21
Example FAT

```
F0 FF FF 03 40 00 05 60 00 07 80 00 09 F0 FF 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Converting Clusters to Logical Sectors

If each cluster has one logical sector, then each logical sector has an entry. However, because the FAT area stores only information about file space, and the first two entries are reserved, the first cluster in the file space is cluster number 2. To convert from a cluster number to a logical sector number, do the following.

1. Subtract 2 from the cluster number.
2. Multiply by the number of sectors in a cluster (1 for the 1.44MB floppy disks).

Thus on media that have 1 sector per cluster, with a 1-sector reserved area, 18 sectors for FAT, and 14 sectors for the root directory (exactly the description of a 1.44M floppy), the logical sector number of the first sector of file space is $0 + 1 + 18 + 14 = 33$. This sector will correspond to FAT cluster 2, logical sector 34 to FAT cluster 3, and so on.

Using the Floppy Disk API

The introduction noted that the Linux device interface is the same as the file manager interface. For example, a Linux process executing in user space can open the floppy disk by executing

```
open("/dev/fd0", O_RDONLY);
```

The process then can use the normal kernel `read()` system call to read the entire disk device as if it were a linear sequence of bytes. The byte sequence is determined by the disk layout. That is, the first bytes in the stream are the bytes in the first logical sector on the disk, then the next block of bytes come from the second logical sector, and so on. Depending on exactly how your laboratory machine is configured, the special file for your **A:** floppy disk drive could have a couple of different names, though it is probably `/dev/fd0`. It also might be named `/dev/fdnH1440`, where *n* is a sequence number, probably zero. You then can read the floppy disk contents as a byte stream. To write the floppy disk, you must run your program with superuser permission. For this exercise, you are to use this device interface as though it is a simplified block device interface. That is, you are to open the file and then read or write its contents as full (512-byte) disk sectors. Use the following functions to read and write the disk.

```
int fd_fid;           /* global variable set by the open() function */
int bps;              /* Bytes per sector from disk geometry */
...
bps = ...;
fid = open(DEV_NAME, O_RDWR);           /* Must be superuser
to
write */
...
int ll_fd_read(int sector_number, char *buffer) {
    int dest, len;
    dest = lseek(fid, sector_number*bps, SEEK_SET);
        if(dest != sector_number*bps) {
            /* Error handling here */
        }
    len = read(fid, buffer, bps);
    if(len != bps) {
        /* Error handling here */
    }
    return len;
}
int ll_fd_write(int sector_number, char *buffer) {
    int dest, len;
    dest = lseek(fid, sector_number*bps, SEEK_SET);
        if(dest != sector_number*bps) {
            /* Error handling here */
        }
    len = write(fid, buffer, bps);
    if(len != bps) {
        /* Error handling here */
    }
    return len;
}
```

The `sector_number` is a logical disk number, `sector_number 0` is the boot record, `sector_number 1` is the location of the first FAT table, and so on, as described in Figure 2.15.

Planning a Solution

You might find it useful to write various supporting routines that can be used to write solutions to the various parts of the exercise. Norman Ramsey, in his Windows NT solution to this problem, suggested implementing the following functions.

```
int scanFatDirectorySector(char *buf, char *name);
struct directoryEntry getEntryFromSector(
    char *buf,
    unsigned index
);
unsigned getFat12Entry(Disk d, unsigned index);
unsigned readFat12Cluster(
    Disk,
    unsigned cluster,
    char *buf
);
```

You also might find it useful to write functions to implement other routines such as

```
int getBootInfo(Disk, BootDescriptor);
```

to extract critical information stored in the boot sector and then return the information in a `BootDescriptor` structure of the following example form.

```
typedef struct BootDescriptor_t *BootDescriptor;
typedef struct BootDescriptor_t {
    unsigned    sectorsPerCluster;
    unsigned    numberOfFATs;
    unsigned    numberOfRootDirEntries;
    unsigned    numberOfLogSectors;
    unsigned    sectorsPerFAT;
    unsigned    numberOfHiddenSectors;
};
```

The `scanFatDirectorySector()` Function

The `scanFatDirectorySector()` function should search a directory in a sector, looking for an entry with a name passed as a parameter. Do not forget that MS-DOS filenames are case-insensitive, so you must match names regardless of case. (Because filenames are stored on the disk in uppercase letters, you can do this by applying `toupper()` to the name for which you are searching). The function should return `-1` if it fails to find a matching directory entry.

Otherwise, it should return the index of the matching entry, which is guaranteed to be in the range 0 to 15.

The `getEntryFromSector()` Function

The `getEntryFromSector()` function should use the index returned by `scanFatDirectorySector()` to retrieve the contents of the entry. Invalid directory entries have filenames that begin with `0x00` (the entry has never been used) or `0xe5` (the entry was previously used but has been released). The directory entry can be represented by the following C structure.

```
struct directoryEntry {
    char fullname[13]; /*name & extension, null-terminated */
    unsigned short year, month, day, hour, min, sec;
    unsigned short firstCluster;
    unsigned long size;
    unsigned readonly:1;
    unsigned hidden:1;
    unsigned system:1;
    unsigned vlabel:1;
    unsigned subdir:1;
    unsigned archive:1;
};
```

You cannot simply do bitwise copies from the directory entry into this structure. Instead, you must unmarshal it and then fill in the fields as appropriate. Integers are little-endian on the disk. Also, the 2-byte field for the time is computed as

$$2048 * \text{hours} + 32 * \text{minutes} + \text{seconds} / 2$$

Thus you must unpack the time field by building the 2-byte value and then performing an integer divide to determine the hour value, the minute value, and the second value. Similarly, the 2-byte date field is encoded as

$$512 * (\text{year} - 1980) + 32 * \text{month} + \text{day}$$

The getFat12Entry() Function

The getFat12Entry() function should take an index (and disk descriptor) and return the unsigned index into a FAT. In the example, the FAT shown in Figure 2.21, getFat12Entry(d, 9), should return 0xffff, which is the logical sector number in the ninth entry in the FAT.

The readFat12Cluster() Function

The readFat12Cluster() function should perform the translation of a cluster to a logical sector number described in the exercise's introduction. It should return zero on success and nonzero on failure.

Following is the header file used for the solution to this exercise.

```
/* This interface is derived from one designed
 * by Norman Ramsey for CSci 413 at Purdue University, 1996
 */
#ifndef FILESYS_H
#define FILESYS_H
#include <windows.h>
#include "..\exercise10.h"
#define DIR_ENTRY_SIZE 32
// typedef struct open_file *Open_File;
typedef struct file_system *FileSystem;
typedef struct BootDescriptor_t *BootDescriptor;
typedef struct BootDescriptor_t {
```

```
    unsigned    sectorsPerCluster;
    unsigned    numberOfFATs;
    unsigned    numberOfRootDirEntries;
    unsigned    numberOfLogSectors;
    unsigned    sectorsPerFAT;
    unsigned    numberOfHiddenSectors;
};

typedef struct directoryEntry {
    char fullname[13]; // Filename and ext, null-terminated
    unsigned short year, month, day, hour, min, sec;
    unsigned short firstCluster;
    unsigned long size;
    unsigned readonly:1;
    unsigned hidden:1;
    unsigned system:1;
    unsigned vlabel:1;
    unsigned subdir:1;
    unsigned archive:1;
};

/* Function prototypes on the File System interface */
// File System Interface
int fd_load(char);
int fd_ls();
int fd_cd(char *directory);
int fd_rm(char *file_name);
// Function prototypes used to implement the file system
unsigned fatOctet(Disk, unsigned);
void findDate(
    unsigned short *year,
    unsigned short *month,
    unsigned short *day,
    unsigned char info[2]
);
void findTime(
    unsigned short* hour,
    unsigned short* min,
    unsigned short* sec,
    unsigned char info[2]
);
int allocSector(Disk, int);
int getBootInfo(Disk, BootDescriptor);
struct directoryEntry getEntryFromSector(char *buf, unsigned index);
unsigned getFat12Entry(Disk d, unsigned index);
unsigned getFATByte(Disk d, unsigned k);
int getFreeDirectoryEntry(char *buf, char *name);
int getRootDirSector();
void intToExtName(char intName[], char extName[]);
int markFATfree(Disk d, int);
int nextRelSector(Disk d, int lastSector);
int putEntryIntoSector(
```

```
    char *buf,  
    struct directoryEntry dirEntry,  
    unsigned index  
);  
unsigned putFat12Entry(Disk d, unsigned index, unsigned val);  
unsigned putFATByte(Disk d, unsigned k, unsigned val);  
unsigned readFat12Cluster(Disk, unsigned cluster, char *buf);  
int scanFatDirectorySector(char *buf, char *name);  
void toBin(unsigned char, int[8]);  
#endif FILESYS_H
```

Notice that you are to read and write a single floppy disk—and with code that references the FAT. For this exercise, you should not attempt to copy the FAT into primary memory, even though not doing so will clearly impact performance negatively. You need to derive your solution by using a floppy disk. Prepare a floppy disk under the usual Windows interface and include a dozen simple files. Your solution should be able to manipulate this floppy disk. You need to write a simple test program to check that each of your commands works. You will likely find it natural to develop this program as you debug each command.